

Parallel Algorithms

Cost Models, Core Primitives, and the Parallel Associative Scan

June 2026

Abstract

Parallelism changes not just how fast an algorithm runs but how we *measure* and *design* it. Sequential analysis asks “how many operations?”; parallel analysis adds “how long is the longest dependency chain?” — the *depth*. This report develops the work–depth cost model, tours the handful of primitives from which most parallel programs are built (map, reduce, scan, compact, sort), and then treats the **parallel associative scan** (prefix sum) in depth: its algorithms, why it is work-efficient, and why it is the secret engine behind everything from radix sort to modern state-space sequence models. Throughout, the lens is the GPU — the machine where these ideas are now most heavily used.

Contents

1	Why parallel algorithms need their own theory	2
2	The cost model: work, depth, and the laws	2
3	The primitive zoo	2
4	Parallel associative scan (prefix sum)	3
4.1	Why associativity is the whole game	3
4.2	Scan vs. reduce: same tree, but every prefix	3
4.3	Why scan matters: it is everywhere	4
4.4	Algorithm 1 — Hillis–Steele (step-efficient)	4
4.5	Algorithm 2 — Blelloch (work-efficient)	4
4.6	Hillis–Steele vs. Blelloch	5
4.7	Segmented scan	5
4.8	Linear recurrences via scan $\Rightarrow$ state-space models	5
4.9	Scan on a real GPU: three levels and decoupled look-back	6
5	Sorting in parallel	6
6	Irregular and graph algorithms	7
7	Mapping to the GPU	7
8	Synthesis	7

1 Why parallel algorithms need their own theory

TL;DR. Sequential cost is one number (total operations). Parallel cost is two: *work* (total operations) and *depth* (the longest chain of dependencies). A good parallel algorithm minimizes depth without blowing up work.

A sequential algorithm has one cost: the number of operations, $T(n)$. The moment you have many processors, a second axis appears — how much of that work is forced to happen *in order*. Two algorithms with identical operation counts can have wildly different parallel runtimes if one is a long dependent chain and the other is a shallow tree. So parallel algorithm design optimizes a different objective: expose independent work, and shorten the critical path.

2 The cost model: work, depth, and the laws

Key idea. **Work** W = total operations (the sequential cost). **Depth** D (a.k.a. span) = longest dependency chain = runtime on *infinitely* many processors. On p processors, runtime is between W/p and roughly $W/p + D$.

Work and depth. Model a computation as a DAG: nodes are operations, edges are data dependencies. **Work** W is the number of nodes; **depth** D is the length of the longest path. W is what you'd pay on one processor; D is the best you could ever do with unlimited processors.

Brent's theorem. On p processors a greedy scheduler achieves

$$T_p \leq \frac{W}{p} + D.$$

Two regimes fall out: when p is small, W/p dominates (you want low **work**); when p is large, D dominates (you want low **depth**). The ideal is **work-efficient** *and* shallow: $W = O(\text{sequential})$ and $D = O(\log n)$.

Work-efficiency. An algorithm is work-efficient if its work matches the best sequential algorithm asymptotically. A common trap is buying low depth with extra work (e.g. an $O(n \log n)$ -work scan); fine on paper, wasteful in practice because the extra work is real energy and real cycles.

Amdahl vs. Gustafson. **Amdahl's law** — if a fraction s of work is serial, speedup is capped at $1/s$ no matter how many processors — is the pessimist's view (fixed problem size). **Gustafson's law** is the optimist's view: as you add processors you also scale the problem, so the parallel part grows and the serial fraction shrinks. Both are true; they answer different questions ("strong" vs. "weak" scaling).

3 The primitive zoo

TL;DR. Most parallel programs are compositions of a few primitives: map, reduce, scan, compact, sort, gather/scatter. Learn these and their work/depth, and you can read almost any GPU algorithm.

Map is embarrassingly parallel (depth 1): elementwise ops, activations. **Reduce** folds all elements with an associative operator via a balanced tree — $O(n)$ work, $O(\log n)$ depth (this is the

Primitive	What it does	Work	Depth
Map	apply f to each element independently	$O(n)$	$O(1)$
Reduce	combine all elements with associative $\oplus$	$O(n)$	$O(\log n)$
Scan	all prefixes under associative $\oplus$	$O(n)$	$O(\log n)$
Compact / filter	keep elements passing a predicate, packed	$O(n)$	$O(\log n)$
Sort	order n keys	$O(n \log n)$	$O(\log^2 n)$
Gather / scatter	indexed read / write	$O(n)$	$O(1)$

Table 1: The core data-parallel primitives. Scan and reduce are the associative backbone; compact and sort are built *on top of* scan.

array-sum kernel: reduce in registers/shared, then across blocks). The rest of this report is mostly about **scan**, because it is the least obvious and the most enabling.

4 Parallel associative scan (prefix sum)

Key idea. **Scan** turns a sequence $x_0, x_1, \dots, x_{n-1}$ and an *associative* operator $\oplus$ into all running totals. Inclusive: $y_i = x_0 \oplus \dots \oplus x_i$. Exclusive: $y_i = x_0 \oplus \dots \oplus x_{i-1}$ (with y_0 the identity). It looks inherently sequential — each prefix seems to need the one before — but associativity lets you compute them all in $O(\log n)$ depth.

4.1 Why associativity is the whole game

The running totals *appear* to force a serial chain: $y_i = y_{i-1} \oplus x_i$. What rescues parallelism is that $\oplus$ is **associative**, so partial results can be regrouped: $(x_0 \oplus x_1) \oplus (x_2 \oplus x_3)$ equals $x_0 \oplus (x_1 \oplus (x_2 \oplus x_3))$. Associativity is exactly the property that lets the hardware reorder the combining into a tree — the same property that makes a parallel *reduction* valid and order-independent. (It is also why a tree sum is deterministic while an atomic-add sum is not: both rely on associativity, but floating-point add is only *approximately* associative, so the grouping that scan/reduce fixes is what makes them reproducible.) The operator need not be commutative — scan respects order — only associative.

4.2 Scan vs. reduce: same tree, but every prefix

A reduction and a scan use the *same* associative operator and the *same* balanced tree, so they are easy to conflate — but they return different things:

- **Reduce** returns *one* value, the total $y = x_0 \oplus x_1 \oplus \dots \oplus x_{n-1}$ (the tree’s *root*).
- **Scan** returns n values, every prefix $y_i = x_0 \oplus \dots \oplus x_i$.

A reduce tree *does* compute many partial results on the way up — but each is the combination of an *aligned, contiguous block* (a subtree), e.g. $x_4 \oplus x_5 \oplus x_6 \oplus x_7$, **not** a prefix. The reduce keeps only the root and throws the rest away. A scan needs the prefix ending at *every* position, and an arbitrary prefix $[0, i]$ is generally *not* one of those aligned block sums — it is a specific combination of several of them.

Key idea. Naively, computing all n prefixes looks like n separate reductions — $O(n^2)$ work. Scan gets it in $O(n)$ by *reusing* the reduce’s partial sums: a reduction (up-sweep) builds the block sums, then a second pass (down-sweep) recombines exactly those sums into every prefix. **Scan = reduce + a redistribution pass.**

This is exactly the structure of Blelloch’s algorithm below: the **up-sweep** *is* a **reduction** (it computes the subtree sums, and at the root the grand total), and the **down-sweep** is the extra pass that pushes those partial sums back down so each leaf receives the combination of block-sums to its left — its prefix. Same asymptotic cost as a reduce ($O(n)$ work, $O(\log n)$ depth), but with a second pass and n outputs instead of one. Put simply: *a reduce discards the partial results; a scan is the algorithm that keeps and reuses them to produce every prefix.*

4.3 Why scan matters: it is everywhere

Scan is the unsung workhorse of parallel computing (Blelloch’s point). A few uses, each of which would otherwise look serial:

- **Stream compaction / filter.** An exclusive scan of a 0/1 “keep” flag gives each surviving element its output index in one shot — the basis of **filter**, sparse-to-dense, and removing nulls.
- **Radix sort.** Counting and placing keys by digit is a series of scans; GPU radix sort is essentially “scan in a loop.”
- **Allocation.** Each task wants c_i slots; an exclusive scan of c_i gives every task the starting offset of its block — parallel memory allocation.
- **Recurrences.** Any first-order linear recurrence becomes a scan (§4.8) — the link to sequence models.
- Line-of-sight, polynomial evaluation, quicksort partitioning, run-length encoding, graph algorithms — all reduce to scans.

4.4 Algorithm 1 — Hillis–Steele (step-efficient)

The simplest parallel scan: at step d , every element adds the element 2^d to its left. After $\lceil \log_2 n \rceil$ steps, every prefix is complete.

```

1 // inclusive scan, in place
2 for d = 0 to ceil(log2(n)) - 1:
3     for all i in parallel:
4         if i >= 2^d:
5             x[i] = x[i] + x[i - 2^d]
```

```

input:    3    1    7    0    4    1    6    3
d=0 (+1):3    4    8    7    4    5    7    9
d=1 (+2):3    4   11   11   12   12   11   14
d=2 (+4):3    4   11   11   15   16   22   25    <- inclusive prefixes
```

Cost: depth $O(\log n)$ (great), but work $O(n \log n)$ — every element updates at every step, so it is *not* work-efficient. It is the right choice *within a warp* (32 lanes, only 5 steps, done with shuffles), where simplicity beats the small extra work.

4.5 Algorithm 2 — Blelloch (work-efficient)

Blelloch’s scan does $O(n)$ work in $O(\log n)$ depth using two passes over a balanced tree — an **up-sweep** (a reduction that builds partial sums) and a **down-sweep** (that distributes them back into prefixes).

```

1 // exclusive scan, in place, n a power of two
2 // --- up-sweep (reduce): build partial sums up the tree ---
3 for d = 0 to log2(n) - 1:
4     for i = 0 to n-1 step 2^(d+1) in parallel:
5         x[i + 2^(d+1) - 1] += x[i + 2^d - 1]
6
7 x[n-1] = identity // seed the exclusive scan
8
9 // --- down-sweep: push partials back down into prefixes ---
10 for d = log2(n) - 1 downto 0:
11     for i = 0 to n-1 step 2^(d+1) in parallel:
12         t = x[i + 2^d - 1]
13         x[i + 2^d - 1] = x[i + 2^(d+1) - 1]
14         x[i + 2^(d+1) - 1] = x[i + 2^(d+1) - 1] + t

```

up-sweep builds a reduction tree (each node = sum of its subtree);
set root = 0; down-sweep swaps-and-adds so each leaf receives the
sum of everything to its left == the exclusive prefix.

Cost: work $O(n)$, depth $O(\log n)$ — work-efficient *and* shallow. Each element is touched a constant number of times per level and there are $2 \log n$ levels total. This is the textbook GPU block-scan.

4.6 Hillis–Steele vs. Blelloch

	Work	Depth	Use when
Hillis–Steele	$O(n \log n)$	$O(\log n)$	small n (a warp); simplicity wins
Blelloch	$O(n)$	$O(\log n)$	large n ; work (energy/cycles) matters

Table 2: Same depth; the difference is total work. On a GPU you typically use Hillis–Steele inside a warp and Blelloch-style across a block.

4.7 Segmented scan

A **segmented scan** runs independent scans over contiguous segments marked by a flag array, in one pass. The trick: define a combined operator on (value, flag) pairs that *resets* the running total at a segment boundary. It stays associative, so it parallelizes exactly like an ordinary scan — and it is how irregular, variable-length work (ragged batches, sparse rows, quicksort sub-partitions) is done data-parallel.

4.8 Linear recurrences via scan $\Rightarrow$ state-space models

Key idea. Any first-order linear recurrence $x_i = a_i x_{i-1} + b_i$ is a scan — because composing affine maps is associative. This is exactly how modern state-space sequence models (S4/S5, Mamba) run their recurrence in parallel on a GPU.

The recurrence $x_i = a_i x_{i-1} + b_i$ looks hopelessly serial. But represent each step as an affine map $f_i(x) = a_i x + b_i$, encoded by the pair (a_i, b_i) . Composing two such maps is itself an affine map:

$$(a_2, b_2) \bullet (a_1, b_1) = (a_2 a_1, a_2 b_1 + b_2),$$

and composition of functions is **associative**. So a scan with the operator $\bullet$ produces, at position i , the composed map from the start through i ; applying it to x_0 gives x_i — all positions in $O(\log n)$ depth.

This is not a curiosity: it is the computational core of **state-space models**. An SSM layer is a linear recurrence $h_t = A_t h_{t-1} + B_t u_t$; training it efficiently on a GPU means evaluating that recurrence in parallel, which is a **parallel associative scan** over the (A_t, B_t) elements. Mamba’s “selective scan” is precisely a hardware-aware segmented associative scan. The same primitive that compacts arrays and drives radix sort is what lets a sequence model avoid the serial RNN bottleneck.

4.9 Scan on a real GPU: three levels and decoupled look-back

TL;DR. Real GPU scan is hierarchical: scan within each warp (shuffles), then across warps in a block (shared memory), then across blocks. The modern single-pass method is *decoupled look-back* (CUB), which overlaps the block-sum propagation with computation.

A device-wide scan composes the primitives:

1. **Warp scan** — 32 lanes via Hillis–Steele with `__shfl_up_sync` (5 steps, no shared memory).
2. **Block scan** — combine warp totals through shared memory, then add each warp’s offset back (Blelloch-style across warps).
3. **Device scan** — scan the per-block totals, then add each block’s offset.

The naive device step needs multiple passes (scan blocks, scan block-sums, add back). The state-of-the-art is **decoupled look-back** (Merrill & Garland, used in **CUB**’s `DeviceScan`): a single pass where each block computes its local sum, publishes it, and “looks back” at predecessors’ published partials to get its prefix — overlapping the dependency propagation with useful work and hitting near memory-bandwidth throughput. For real code, call `cub::DeviceScan / thrust::exclusive_scan`; hand-rolling rarely beats it.

5 Sorting in parallel

TL;DR. Parallel sorts trade the clean $O(n \log n)$ comparison bound for predictable, data-parallel structure. Radix sort (scan-based) dominates on GPUs for integer keys; bitonic and sample sort are the comparison-based options.

- **Radix sort** — process keys digit by digit; each pass is a count + **scan** + scatter. No comparisons, fully data-parallel, $O(n)$ per pass — the GPU default for fixed-width keys, and a direct application of scan.
- **Bitonic sort** — a sorting network of compare-exchange steps with a fixed, data-independent pattern; $O(n \log^2 n)$ work but branch-free and predictable, good for small/medium arrays and hardware.
- **Sample sort / merge sort** — pick splitters to bucket data across processors, sort buckets locally; the basis of large distributed sorts.

6 Irregular and graph algorithms

TL;DR. Pointers and graphs break the neat array model. The key tricks are *pointer jumping* (halve a chain’s length each step) and framing graph steps as sparse matrix–vector products.

List ranking via pointer jumping. Finding each node’s distance to the end of a linked list seems serial, but **pointer jumping** — repeatedly replace each node’s successor with its successor’s successor while summing distances — collapses the list in $O(\log n)$ steps. It is the canonical “break the serial chain” technique and underlies many tree/graph algorithms (Euler tours, connectivity).

Graph traversal. BFS parallelizes level-by-level (process a whole frontier at once); connected components via label propagation / pointer jumping. Many graph steps are sparse matrix–vector products (SpMV), which is why GPU graph frameworks (Gunrock) and “graph-as-linear-algebra” (GraphBLAS) exist — and why these kernels are memory-bound (irregular, hard-to-coalesce access).

7 Mapping to the GPU

TL;DR. The GPU hierarchy (warp $\rightarrow$ block $\rightarrow$ grid) is a physical realization of the work–depth model. Use library primitives (CUB/Thrust) unless you have a reason not to — they encode the hierarchical, work-efficient versions.

The three-level scan above is the template for almost everything: do the work-inefficient-but-simple thing at the smallest level (a warp, where the log factor is tiny), and the work-efficient thing across larger levels. Reductions, scans, sorts, and histograms all follow this warp $\rightarrow$ block $\rightarrow$ grid decomposition, and the depth at each level is the $O(\log(\text{width}))$ of that level. **CUB** (device/block-/warp primitives) and **Thrust** (STL-like) ship the tuned versions; reach for a custom kernel only for fusion or a shape the library doesn’t serve.

8 Synthesis

Idea	Why it matters	Shows up as
Work vs. depth	two costs, not one; shorten the critical path	Brent’s theorem
Associativity	lets partial results regroup into a tree	reduce, scan, SSMs
Scan	makes “serial-looking” problems parallel	compact, sort, recurrences
Work-efficiency	low depth must not cost extra work	Blelloch over Hillis–Steele
Hierarchy	warp/block/grid = the cost model in silicon	CUB/Thrust

Bottom line. The shift from sequential to parallel thinking is the shift from *counting operations* to *also counting dependencies*. The central enabling property is **associativity**, and the central enabling primitive is the **parallel associative scan** — $O(n)$ work, $O(\log n)$ depth, and a remarkable range: array compaction, radix sort, memory allocation, and the linear recurrences at the heart of modern state-space sequence models. If you internalize work–depth, associativity, and scan, most parallel algorithms read as variations on a theme.

Selected references. Blelloch, *Prefix Sums and Their Applications*, 1990; JáJá, *An Introduction to Parallel Algorithms*, 1992; Brent, *The Parallel Evaluation of General Arithmetic Expressions*, 1974; Hillis & Steele, *Data Parallel Algorithms*, CACM 1986; Merrill & Garland, *Single-pass Parallel Prefix Scan with Decoupled Look-back*, 2016; Gu & Dao, *Mamba: Linear-Time Sequence Modeling with Selective State Spaces*, 2023; Harris et al., *Parallel Prefix Sum (Scan) with CUDA*, GPU Gems 3.